

Задачи към Упр1 - Python

Пример 1:

```
#!/usr/bin/python
```

```
counter = 100 # An integer assignment
```

```
miles = 1000.0 # A floating point
```

```
name = "John" # A string
```

```
print (counter)
```

```
print (miles)
```

```
print (name)
```

Резултат:

100

1000.0

John

Пример 2:

```
a=b=c=1
```

```
print (a) //извеждане на стойността на a
```

```
print (a, b, c) //извеждане на стойността на a, b, c
```

Резултат:

```
1 // принтиране на a
```

```
1 1 1 // принтиране на едновременно на трите стойности
```

Пример 3:

```
a,b,c = 1,2,"john "
```

```
print (a, b, c)
```

Резултат:

1 2 john

Пример 4: Изрази за присвояване

example.py

```
#!/usr/bin/env python
```

```
# Computer the value of a block of stock
```

```
shares= 150
```

```
price= 3 + 5.0/8.0
```

```
value= shares * price
```

```
print (value)
```

Резултат: 543.75

Пример 5: Да се създаде програма за осредняване на резултатите от 2 изпита:

Вариант 1

```
import math, cmath
```

```
score1, score2=(input ("Discipline assessment score1 and score2=")).split()
```

```
score1, score2=[float(score1), float(score2)]
```

```
average=(score1 + score2) / 2
```

```
print ('DisAsses 1={0}, DisAsses 2={1}, Result={2}'.format(score1, score2, average))
```

main.py	  	Shell
<pre>1 import math, cmath 2 score1, score2=(input ("Discipline assessment score1 and score2=")).split() 3 score1, score2=[float(score1), float(score2)] 4 average=(score1 + score2) / 2 5 print ('DisAsses 1={0}, DisAsses 2={1}, Result={2}'.format(score1, score2, average))</pre>		<pre>Discipline assessment score1 and score2=2.5 3.5 DisAsses 1=2.5, DisAsses 2=3.5, Result=3.0 ></pre>

Вариант 2

```
# avg.py
```

```
score1 = float(input("Discipline assessment 1: "))
```

```
score2 = float(input("Discipline assessment 2: "))
```

```
average=(score1 + score2) / 2
```

```
print ("The average of the scores is:", average)
```

Вход: Discipline assessment 1: 2.5, Discipline assessment 2: 3.5

Изход: The average of the scores is: 3

main.py	  	Shell
<pre>1 score1 = float(input("Discipline assessment 1: ")) 2 score2 = float(input("Discipline assessment 2: ")) 3 average=(score1 + score2) / 2 4 print ("The average of the scores is:", average)</pre>		<pre>Discipline assessment 1: 2.5 Discipline assessment 2: 3.5 The average of the scores is: 3.0 ></pre>

Пример 6:

```
#Деклариране на константи във файл наречен - constant.py
PI = 3.14
GRAVITY = 9.8
# Импортиране в основния файл на тези константи
#inside main.py we import the constants import constant
print(constant.PI)
print(constant.GRAVITY)
```

Пример 7:

Изчисляване на индекс на телесна маса в Python

```
#Define the constants
METER = 100
#Read the inputs from user
height = float(input("Enter your height in Centimeters: "))
weight = float(input("Enter your weight in Kg: "))
temp = height / METER
#Calculate the BMI
bmi = weight / (temp*temp)
#Display the result
print("Your Body Mass Index is: ", "%d"%(bmi))
Резултат:
C:\Python\programs>python program.py
Enter your height in Centimeters: 105
Enter your weight in Kg: 69
Your Body Mass Index is: 62
```

Операторът break в Python прекратява текущия цикъл и възобновява изпълнението при следващия оператор, точно както традиционния break в програмният език C.

Най-честата употреба за прекъсване е, когато се задейства някакво външно условие, изискващо прибързано излизане от цикъл. Операторът break може да се използва както в цикли while, така и във for.

```
#!/usr/bin/python
for letter in 'Python':
    if letter == 'h':
        break
```

```
print ("Current Letter :", letter)
```

Результат:

Current Letter : h

Пример 8:

```
#!/usr/bin/python
```

```
for letter in 'Python':
```

```
    # First Example
```

```
    if letter == 'h':
```

```
        break
```

```
    else:
```

```
        print ("Current Letter :", letter)
```

```
var = 10
```

```
    # Second Example
```

```
while var > 0:
```

```
    var = var -1
```

```
    if var == 5:
```

```
        break
```

```
    else:
```

```
        print ("Current Letter :", var)
```

```
print ("Good bye!")
```

Результат:

Current Letter : P

Current Letter : y

Current Letter : t

Current Letter : 9

Current Letter : 8

Current Letter : 7

Current Letter : 6

Good bye!

main.py	  	Shell
<pre>1 # Online Python compiler (interpreter) to run Python online. 2 # Write Python 3 code in this online editor and run it. 3 for letter in 'Python': 4 # First Example 5 if letter == 'h': 6 break 7 else: 8 print ("Current Letter :", letter) 9 var = 10 10 # Second Example 11 while var > 0: 12 var = var -1 13 if var == 5: 14 break 15 else: 16 print ("Current Letter :", var) 17 print ("Good bye!")</pre>		<pre>Current Letter : P Current Letter : y Current Letter : t Current Letter : 9 Current Letter : 8 Current Letter : 7 Current Letter : 6 Good bye! ></pre>

Операторът continue в Python връща контролата в началото, в сравнение със цикъла while. Инструкцията за продължаване отхвърля всички останали изрази в текущата итерация на цикъла и премества контролата обратно в началото на цикъла.

Инструкцията за продължаване може да се използва както в цикли while, така и в for.

Пример 9:

```
#!/usr/bin/python
for letter in 'Python':
    # First Example
    if letter == 'h':
        continue
    else:
        print ("Current Letter :", letter)
var = 10
# Second Example
while var > 0:
    var = var -1
    if var == 5:
        continue
    else:
        print ("Current Letter :", var)
```

```
print ("Good bye!")
```

Резултат:

Current Letter : P

Current Letter : y

Current Letter : t

Current Letter : o

Current Letter : n

Current Letter : 9

Current Letter : 8

Current Letter : 7

Current Letter : 6

Current Letter : 4

Current Letter : 3

Current Letter : 2

Current Letter : 1

Current Letter : 0

Good bye!

main.py	Run	Shell
<pre>1 # Online Python compiler (interpreter) to run Python online. 2 # Write Python 3 code in this online editor and run it. 3> for letter in 'Python': 4 # First Example 5> if letter == 'h': 6 continue 7> else: 8 print ("Current Letter :", letter) 9 var = 10 10 # Second Example 11> while var > 0: 12 var = var -1 13> if var == 5: 14 continue 15> else: 16 print ("Current Letter :", var) 17 print ("Good bye!")</pre>		<pre>Current Letter : P Current Letter : y Current Letter : t Current Letter : o Current Letter : n Current Letter : 9 Current Letter : 8 Current Letter : 7 Current Letter : 6 Current Letter : 4 Current Letter : 3 Current Letter : 2 Current Letter : 1 Current Letter : 0 Good bye! ></pre>

Пример 10:

Да се напише задача, която търси прости числа в интервала от 10 до 20, като се използва комбинацията от оператор else с оператор for:

```
#!/usr/bin/python
```

```
for num in range(10,20): #to iterate between 10 to 20
```

```

for i in range(2,num): #to iterate on the factors of the number
    if num%i == 0:
        j=num/i
        print ('%d = %d * %d' % (num,i,j))
        break
    else: print (num, 'is a prime number')

```

Резултат:

```

10 = 2 * 5
11 is a prime number
12 = 2 * 6
13 is a prime number
14 = 2 * 7
15 = 3 * 5
16 = 2 * 8
17 is a prime number
18 = 2 * 9
19 is a prime number

```

main.py	Run	Shell
<pre> 1• for num in range(10,20): #to iterate between 10 to 20 2• for i in range(2,num): #to iterate on the factors of the number 3• if num%i == 0: 4 j=num/i 5 print ('%d = %d * %d' % (num,i,j)) 6 break 7 else: print (num, 'is a prime number') 8 </pre>		<pre> 10 = 2 * 5 11 is a prime number 12 = 2 * 6 13 is a prime number 14 = 2 * 7 15 = 3 * 5 16 = 2 * 8 17 is a prime number 18 = 2 * 9 19 is a prime number > </pre>

По подобен начин можете да използвате оператор else с цикъл while.

pass Statement:

Операторът pass в Python се използва, когато израз се изисква синтактично, но не искате да се изпълнява никаква команда или код.

Операцията pass е нулева операция; нищо не се случва, когато се изпълнява.

Пропускът е полезен и на места, където вашият код в крайна сметка ще отиде, но все още не е написан:

```
#!/usr/bin/python
```

```
for letter in 'Python':
```

```
    if letter == 'h':
```

```

pass
else:
    print ("Current Letter :", letter)
print ("Good bye!")

```

Резултат ще е следния:

```

Current Letter : P
Current Letter : y
Current Letter : t
Current Letter : o
Current Letter : n
Good bye!

```

main.py	  	Shell
<pre> 1 # Online Python compiler (interpreter) to run Python online. 2 # Write Python 3 code in this online editor and run it. 3 for letter in 'Python': 4 if letter == 'h': 5 pass 6 else: 7 print ("Current Letter :", letter) 8 print ("Good bye!") </pre>		<pre> Current Letter : P Current Letter : y Current Letter : t Current Letter : o Current Letter : n Good bye! > </pre>

Пример 11: ПОКАЗВАНЕ НА ПРОСТИ ЧИСЛА

#Програма на Python с която се извеждат всички прости числа в интервала от 900 до 1000

```

lower = 900
upper = 1000
print("Prime numbers between", lower, "and", upper, "are:")
for num in range(lower, upper + 1):
    if num > 1:
        for i in range(2, num):
            if (num % i) == 0: break
        else: print(num)

```

Резултат:

```

Prime numbers between 900 and 1000 are: 907
911
919

```


929
937
941
947
953
967
971
977
983
991
997

main.py	Shell
<pre>1 # Online Python compiler (interpreter) to run Python online. 2 # Write Python 3 code in this online editor and run it. 3 lower = 900 4 upper = 1000 5 print("Prime numbers between", lower, "and", upper, "are:") 6 for num in range(lower, upper + 1): 7 if num > 1: 8 for i in range(2, num): 9 if (num % i) == 0: break 10 else: print(num)</pre>	<pre>Prime numbers between 900 and 1000 are: 907 911 919 929 937 941 947 953 967 971 977 983 991 997</pre>

Прости числа

Простото число е цяло число, по-голямо от 1, което се дели на 1 и на себе си. Първите няколко прости числа са:

2, 3, 5, 7, 11, 13, 17, 19, 23 и 29.

За да се провери дали едно число, например num е просто, първо се разделя на 2.

Ако резултата от $\text{num} \% 2 = 0$,

то num не е просто.

Иначе е просто число.

Ще се демонстрират 2 варианта на задачата

Пример 12: (using for statement)

To display all the prime numbers within a given interval [Lower, Upper]

```

lower = int(input("Enter Lower Limit: "))
upper = int(input("Enter Upper Limit: "))
print("Prime numbers between", lower, "and", upper, "are:")
for num in range(lower, upper + 1):
    if num > 1:
        for i in range(2, num):
            if (num % i) == 0: break
        else: print(num)

```

Резултат:

Enter Lower Limit: 20

Enter Upper Limit: 50

Prime numbers between 20 and 50 are:

23

29

31

37

41

43

47

>>>

main.py	Run	Shell
1 # Online Python compiler (interpreter) to run Python online.		Enter Lower Limit: 20
2 # Write Python 3 code in this online editor and run it.		Enter Upper Limit: 50
3 # To display all the prime numbers within a given interval [Lower, Upper]		Prime numbers between 20 and 50 are:
4 lower = int(input("Enter Lower Limit: "))		23
5 upper = int(input("Enter Upper Limit: "))		29
6 print("Prime numbers between", lower, "and", upper, "are:")		31
7 for num in range(lower, upper + 1):		37
8 if num > 1:		41
9 for i in range(2, num):		43
10 if (num % i) == 0: break		47
11 else: print(num)		>

Пример 13: (using while statement)

Python program to display all the prime numbers within a given interval [Lower, Upper] using while statement

```
lower = int(input("Enter Lower Limit: "))
```

```
upper = int(input("Enter Upper Limit: "))
print("Prime numbers between", lower, "and", upper, "are:")
num=lower
while(num<=upper):
    if num > 1:
        for i in range(2, num):
            if (num % i) == 0: break
        else: print(num)
        num=num+1
```

Результат:

Enter Lower Limit: 50

Enter Upper Limit: 100

Prime numbers between 50 and 100 are:

53

59

61

67

71

73

79

83

89

97

>>>

main.py	Shell
1 # Online Python compiler (interpreter) to run Python online.	Enter Lower Limit: 50
2 # Write Python 3 code in this online editor and run it.	Enter Upper Limit: 100
3 lower = int(input("Enter Lower Limit: "))	Prime numbers between 50 and 100 are:
4 upper = int(input("Enter Upper Limit: "))	53
5 print("Prime numbers between", lower, "and", upper, "are:")	59
6 num=lower	61
7 while(num<=upper):	67
8 if num > 1:	71
9 for i in range(2, num):	73
10 if (num % i) == 0: break	79
11 else: print(num)	83
12 num=num+1	89
13	97

Пример 14: РЕШАВАНЕ НА КВАДРАТИЧНО УРАВНЕНИЕ

В алгебрато квадратно уравнение е всяко уравнение от вида $ax^2 + bx + c = 0$, където x е неизвестно, а a , b и c са известни числа. Числата a , b и c са коефициентите на уравнението.

Примери: $x^2 + 5x + 6 = 0$, $4x^2 + 5x + 8 = 0$.

Стойностите на x , които удовлетворяват уравнението, се наричат решения или корени на уравнението. Квадратното уравнение винаги има два корена. Може да се намери чрез изчисляване на $d = b^2 - 4ac$. Стойността на d е нула или $+ve$ или $-ve$.

Ако $d = 0$, тогава корените са реални и равни. Те са корен1 = $-b/(2a)$ и корен2 = $-b/(2a)$.

Ако $d > 0$, тогава корените са реални и различни. Те са корен1 = $-b + \sqrt{d}/(2a)$ и корен2 = $-b - \sqrt{d}/(2a)$.

Ако $d < 0$, тогава корените са комплексни или имагинерни. Корените са от вида $p + iq$ и $p - iq$, където p е реалната част от корена, а q е въображаемата част от корена. Реалната част на корена е $p = -b/(2a)$, а имагинерната част е $q = \sqrt{-d}/(2a)$.

Вариант 1:

main.py	  	Shell
<pre>1 # Online Python compiler (interpreter) to run Python online. 2 # Write Python 3 code in this online editor and run it. 3 import math,cmath 4 a,b,c= (input("Enter a, b and c: ")).split() 5 a,b,c =[int(a),int(b),int(c)] 6 d = (b**2) - (4*a*c) 7 if d==0: 8 sol1 = -b/(2*a) 9 sol2 = -b/(2*a) 10 print("Roots are real and equal") 11 elif d>0: 12 sol1 = (-b- math.sqrt(d))/(2*a) 13 sol2 = (-b+math.sqrt(d))/(2*a) 14 print("Roots are real and different") 15 elif d<0: 16 sol1 = (-b-cmath.sqrt(d))/(2*a) 17 sol2 = (-b+cmath.sqrt(d))/(2*a) 18 print("Roots are imaginary") 19 print('{0} and {1}'.format(sol1,sol2))</pre>		<pre>Enter a, b and c: 1 4 4 Roots are real and equal -2.0 and -2.0 > </pre>

Вариант 2

Finding the roots of quadratic equation $ax^2 + bx + c = 0$ # import math and complex math modules

```
import math,cmath
```

```
a,b,c= (input("Enter a, b and c: ")).split()
```

```
a,b,c =[int(a),int(b),int(c)]
```

```
d = (b**2) - (4*a*c)
```

```
if d==0:
```

```
    sol1 = -b/(2*a)
```

```
    sol2 = -b/(2*a)
```

```
    print("Roots are real and equal")
```

```
elif d>0:
```

```
    sol1 = (-b- math.sqrt(d))/(2*a)
```

```
    sol2 = (-b+math.sqrt(d))/(2*a)
```

```
    print("Roots are real and different")
```

```
elif d<0:
```

```
    sol1 = (-b-cmath.sqrt(d))/(2*a)
```

```
    sol2 = (-b+cmath.sqrt(d))/(2*a)
```

```
print("Roots are imaginary")
print('{0} and {1}'.format(sol1,sol2))
```

Резултат:

Enter a, b and c: 1 4 4 Roots are real and equal

-2.0 and -2.0

>>>

Enter a, b and c: 1 5 6 Roots are real and different

-3.0 and -2.0

>>>

Enter a, b and c: 1 2 3 Roots are imaginary

(-1-1.4142135623730951j) and (-1+1.4142135623730951j)

main.py	  	Shell
<pre>1 # Online Python compiler (interpreter) to run Python online. 2 # Write Python 3 code in this online editor and run it. 3 import math,cmath 4 a,b,c= (input("Enter a, b and c: ")).split() 5 a,b,c =[int(a),int(b),int(c)] 6 d = (b**2) - (4*a*c) 7 if d==0: 8 sol1 = -b/(2*a) 9 sol2 = -b/(2*a) 10 print("Roots are real and equal") 11 elif d>0: 12 sol1 = (-b- math.sqrt(d))/(2*a) 13 sol2 = (-b+math.sqrt(d))/(2*a) 14 print("Roots are real and different") 15 elif d<0: 16 sol1 = (-b-cmath.sqrt(d))/(2*a) 17 sol2 = (-b+cmath.sqrt(d))/(2*a) 18 print("Roots are imaginary") 19 print('{0} and {1}'.format(sol1,sol2))</pre>		<pre>Enter a, b and c: 1 4 4 Roots are real and equal -2.0 and -2.0 > </pre>

Задачи за самостоятелна работа:

Зад. 1. Да се създаде приложение, което да имплементира Аритметични оператори от Таблица 1

Таблица 1. Аритметични оператори

оператор	описание	пример
+ Addition	Добавя стойности от двете страни на оператора.	$a + b = 30$
- Subtraction	Изважда десния операнд от левия операнд.	$a - b = -10$
* Multiplication	Умножава стойностите от двете страни на оператора.	$a * b = 200$
/ Division	Разделя левия операнд на десен операнд.	$b / a = 2$
% Modulus	Разделя левия операнд на десен операнд и връща остатъка.	$b \% a = 0$
** Exponent	Извършва експоненциално изчисление на операторите	$a^{**}b = 10$ на степен 20
// Floor Division	Деление на операндите, при което резултатът е частното, в което се премахват цифрите след десетичната запетая. Но ако един от операндите е отрицателен, резултатът е занижен, т.е. закръгля се от нула (към отрицателна	$9//2 = 4$, $9.0//2.0=4.0$, $-11//3 = -4$, $-11.0//3 = -4.0$

	безкрайност)	
--	--------------	--

Зад. 2. Да се създаде приложение, което да имплементира Оператори за сравнение от Таблица 2
Таблица 2. Оператори за сравнение

оператор	описание	пример
==	Ако стойностите на два операнда са равни, тогава условието става вярно.	(a == b) не е true
!=	Ако стойностите на два операнда не са равни, тогава условието става вярно.	(a != b) е true
<>	Ако стойностите на два операнда не са равни, тогава условието става вярно.	(a <> b) е true Това е подобно на оператора !=
>	Ако стойността на левия операнд е по-голяма от стойността на десния операнд, тогава условието става вярно.	(a > b) не е true
<	Ако стойността на левия операнд е по-малка от стойността на десния операнд, тогава условието става вярно.	(a < b) е true
>=	Ако стойността на левия операнд е	(a >= b) не е

	по-голяма или равна на стойността на десния операнд, тогава условието става вярно.	true
<=	Ако стойността на левия операнд е по-малка или равна на стойността на десния операнд, тогава условието става вярно.	(a <= b) е true

Зад. 3. Да се създаде приложение, което да имплементира Оператори за присвояване от Таблица 2
Таблица 3. Оператори за присвояване

оператор	описание	пример
=	Присвоява стойности от десните операнди на левия операнд	c = a + b присвоява стойност на a + b на c
+= Add AND	Той добавя десния операнд към левия операнд и присвоява резултата на левия операнд	c += a е еквивалентно на c = c + a
-= Subtract AND	Той изважда десния операнд от левия операнд и присвоява резултата на левия операнд	c -= a е еквивалентно на c = c - a
*= Multiply AND	Той умножава десния операнд с левия операнд и присвоява резултата на левия операнд	c *= a е еквивалентно на c = c * a

/= Divide AND	Той разделя левия операнд с десния операнд и присвоява резултата на левия операнд	$c /= a$ е еквивалентно на $c = c / a$
%= Modulus AND	Той взема модул, използвайки два операнда и присвоява резултата на левия операнд	$c \% = a$ е еквивалентно на $c = c \% a$
**= Exponent AND	Извършва експоненциално (мощност) изчисление на оператори и присвоява стойност на левия операнд	$c ** = a$ е еквивалентно на $c = c ** a$
//= Floor Division	Той извършва разделяне на етажа на операторите и присвоява стойност на левия операнд	$c //= a$ е еквивалентно на $c = c // a$

Зад. 4. Да се създаде приложение, което да имплементира Побитови оператори от Таблица 4

Таблица 4. Побитови оператори

оператор	описание	пример
& Binary AND	Операторът копира a bit към резултата, ако съществува и в двата операнда	(a & b) (означава 0000 1100)
Binary OR	Копира a bit, ако съществува в който и да е	(a b) = 61 (означава

	операнд.	0011 1101)
$\wedge$ Binary XOR	Той копира бита, ако е зададен в един операнд, но не и в двата.	$(a \wedge b) = 49$ (означава 0011 0001)
$\sim$ Binary Ones Complement	Той е унарен и има ефект на „обръщане“ на битове.	$(\sim a) = -61$ (означава 1100 0011 във формата за допълване на 2 поради двоично число със знак.)
$\ll$ Binary Left Shift	Стойността на левия операнд се премества наляво с броя битове,	$a \ll 2 = 240$ (означава

	посочени от десния операнд.	1111 0000)
>> Binary Right Shift	Стойността на левия операнд се премества надясно с броя битове, посочени от десния операнд.	a >> 2 = 15 (означава 0000 1111)

Зад. 5. Да се създаде приложение, което да имплементира Логически оператори от Таблица 5

Таблица 5. Логически оператори

оператор	описание	пример
and Logical	Ако и двата операнда са верни, тогава условието става	(a and b) e true
or Logical OR	Ако някой от двата операнда е различен от нула, тогава условието става вярно.	(a or b) e true
not Logical NOT	Използва се за обръщане на логическото състояние на неговия операнд.	Not (a and b) e false

Задължителна домашна работа

Пример 6: Да се дефинират две променливи x и y. Да се намери сумата и разликата от двете променливи.

Пример 7: Да се дефинират две променливи x и y.

Да се разменят стойностите им - т.е. стойността, която в момента се съхранява в x, да бъде в y, а стойността, която в момента е в y, да се съхранява в x.